Lab Experiment 7: CI/CD using Jenkins, GitHub and Docker Hub

1. Aim

To design and implement a complete CI/CD pipeline using Jenkins, integrating source code from GitHub, and building & pushing Docker images to Docker Hub.

2. Objectives

- Understand CI/CD workflow using Jenkins (GUI-based tool)
- Create a structured GitHub repository with application + Jenkinsfile
- Build Docker images from source code
- Securely store Docker Hub credentials in Jenkins
- Automate build & push process using webhook triggers
- Use same host (Docker) as Jenkins agent

3. Theory

What is Jenkins?
Jenkins is a web-based GUI automation server used to:
* Build applications
* Test code
* Deploy software

It provides:
* Dashboard (browser-based UI)
* Plugin ecosystem (GitHub, Docker, etc.)
* Pipeline as Code using Jenkinsfile

What is CI/CD?
* Continuous Integration (CI): Code is automatically built and tested after each commit
* Continuous Deployment (CD): Built artifacts (Docker images) are automatically delivered/deployed

Workflow Overview
Developer -> GitHub -> Webhook -> Jenkins Build -> Docker Hub

4. Prerequisites

- Docker & Docker Compose installed

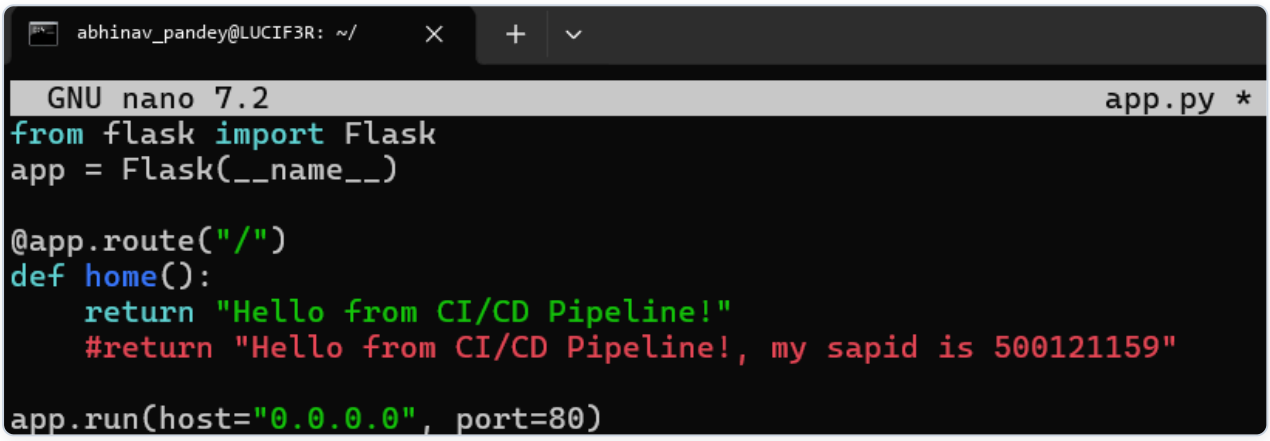
- GitHub account
- Docker Hub account
- Basic Linux command knowledge

5. Part A: GitHub Repository Setup (Source Code + Build Definition)

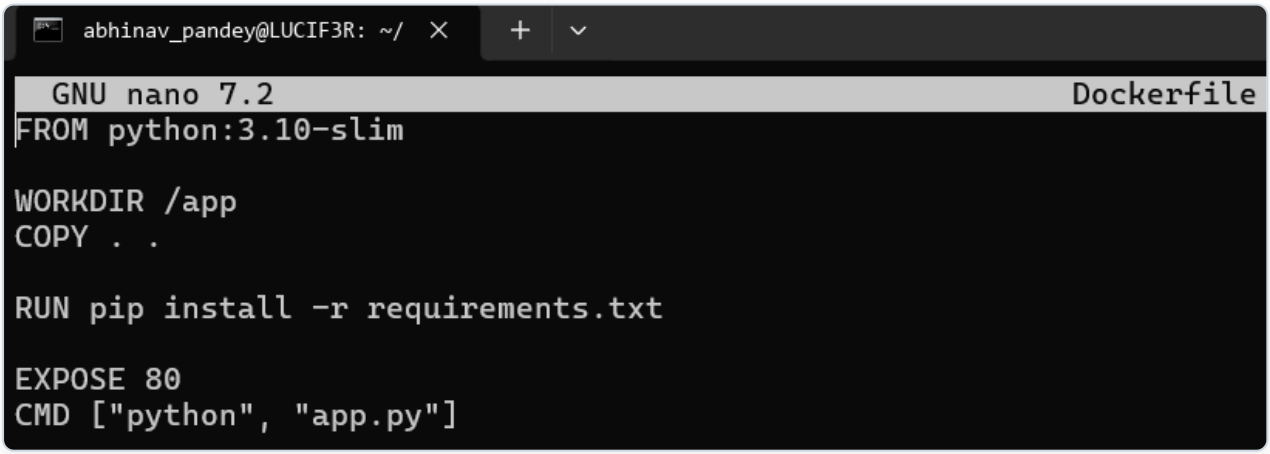
5.1 Create Repository

Create a repository on GitHub: my-app

5.2 Application Code



5.3 Dockerfile (Build Process)



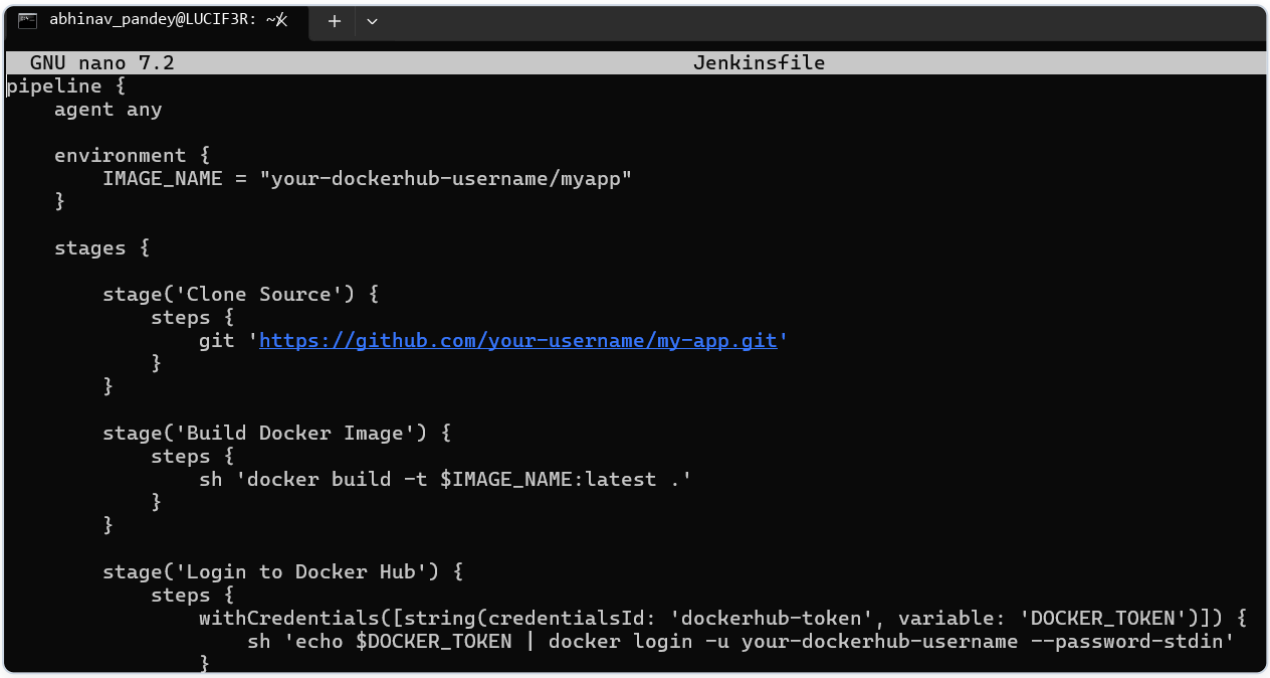
Build Process Explanation

1. Source code pushed to GitHub
2. Jenkins pulls code
3. Dockerfile:

* Creates environment

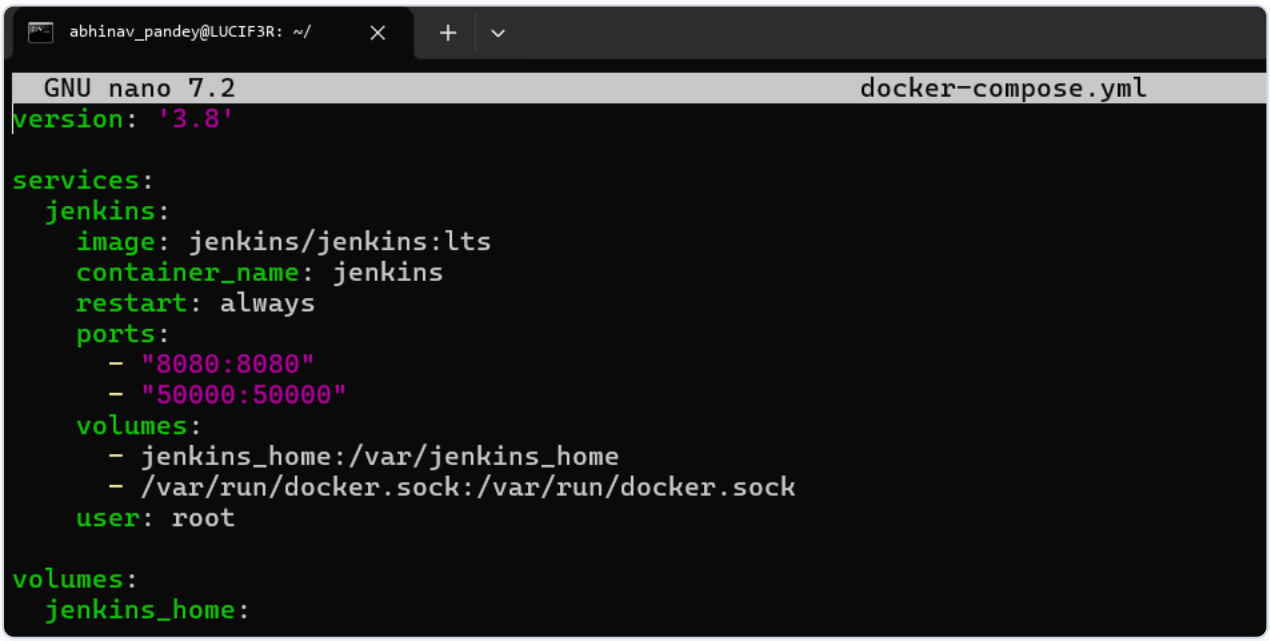
* Installs dependencies
* Packages app
4. Output Docker Image

5.4 Jenkinsfile (Pipeline Definition in GitHub)



6. Part B: Jenkins Setup using Docker (Persistent Configuration)

6.1 Create Docker Compose File



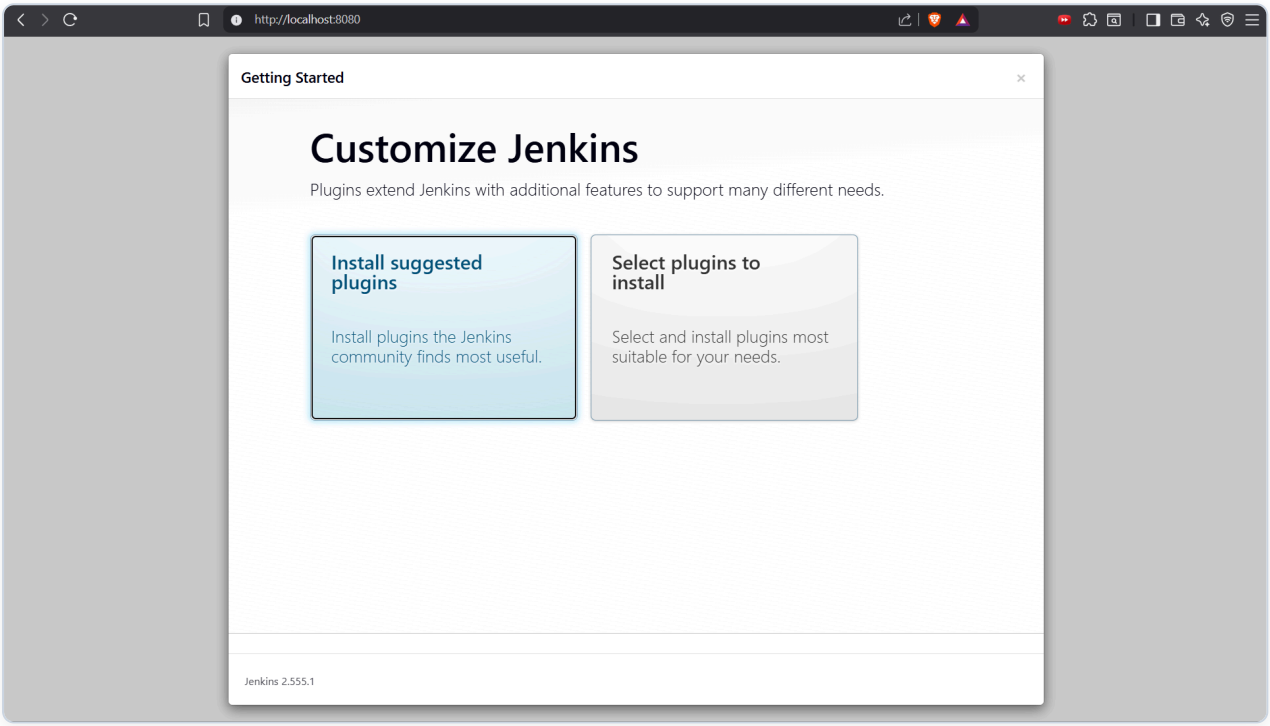
6.2 Start Jenkins

```
abhinav_pandey@LUCIF3R:      ~/my-app$ docker compose up -d
WARN[0000] /home/abhinav_pandey/my-app/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, plea
se remove it to avoid potential confusion
[+] up 1/1
 ✔ Container jenkins Started                                                                                    0.3s
```

Access: http://localhost:8080

6.3 Unlock Jenkins

docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword



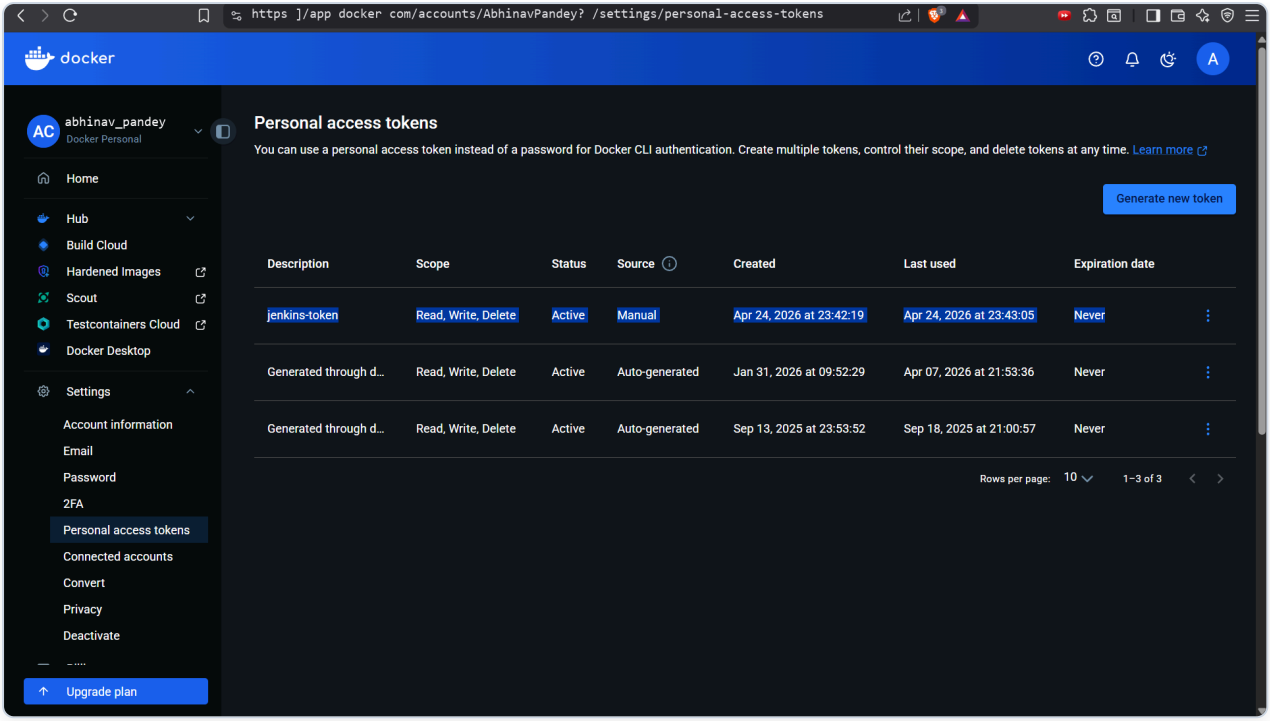
6.4 Initial Setup

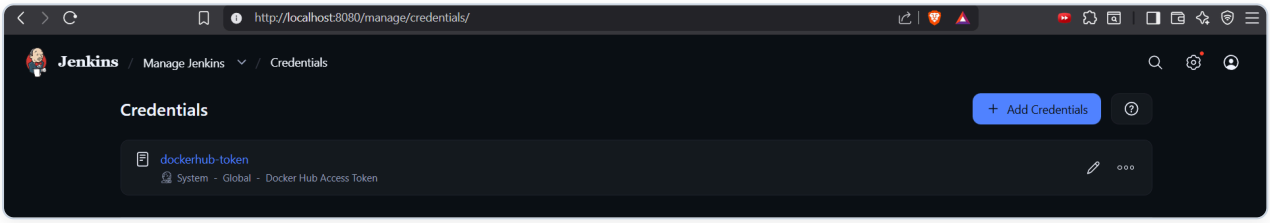
- Install suggested plugins
- Create admin user

7. Part C: Jenkins Configuration

7.1 Add Docker Hub Credentials

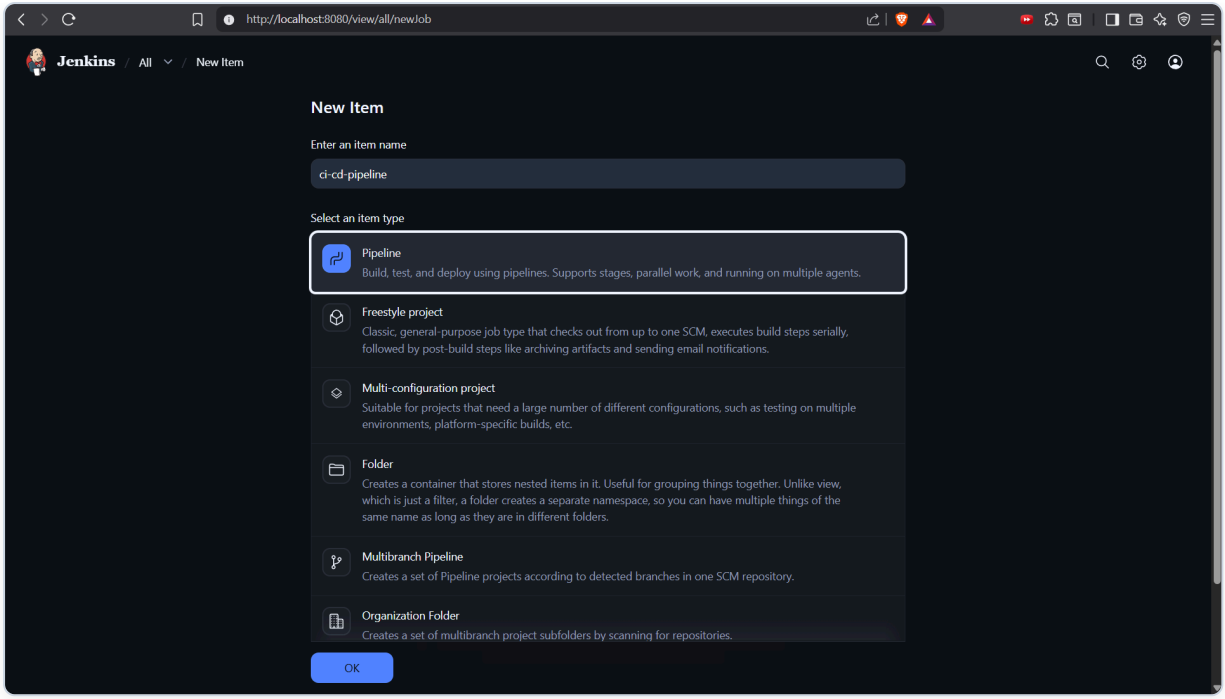
Path: Manage Jenkins -> Credentials -> Add Credentials
* Type: Secret Text
* ID: dockerhub-token
* Value: Docker Hub Access Token





7.2 Create Pipeline Job

1. New Item -> Pipeline
2. Name: ci-cd-pipeline
3. Configure:
   * Pipeline script from SCM
   * SCM: Git
   * Repo URL: your GitHub repo
   * Script Path: Jenkinsfile



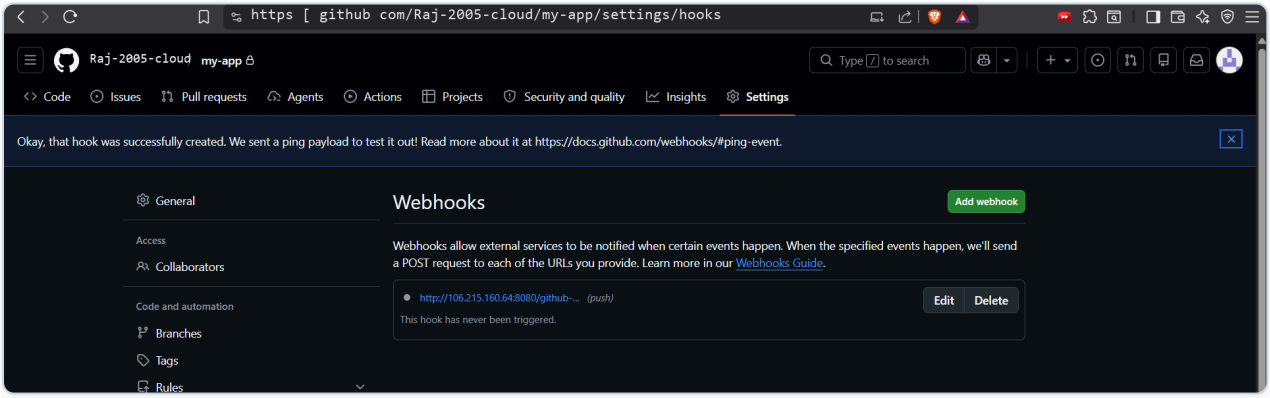
8. Part D: GitHub Webhook Integration

8.1 Configure Webhook

In GitHub: Settings -> Webhooks -> Add Webhook

* Payload URL: http://:8080/github-webhook/

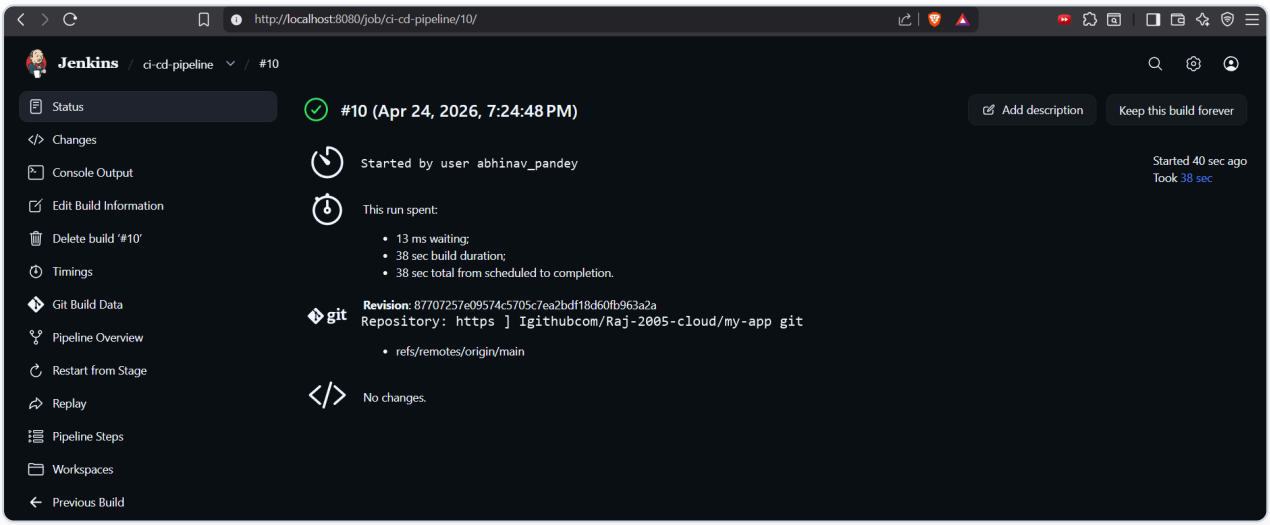
* Events: Push events

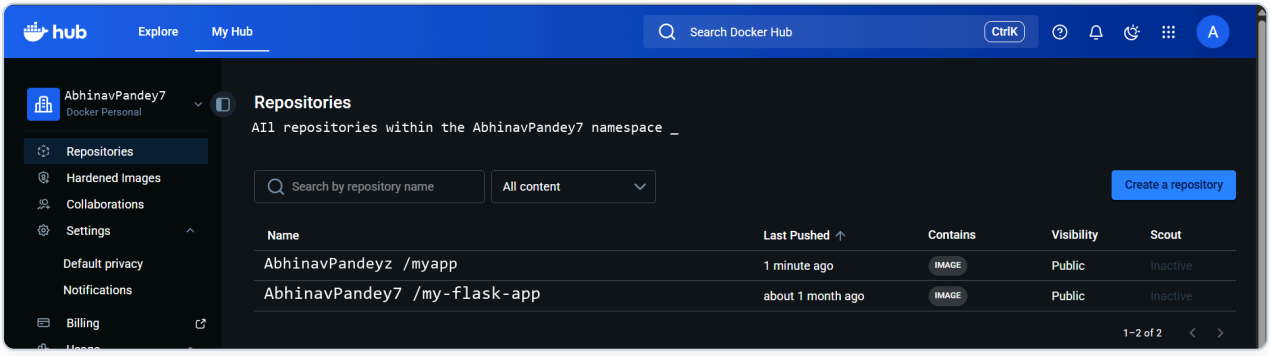


DEMO

```
abhinav_pandey@LUCIF3R:          ~/my-app$ docker exec -it jenkins bash
root@c30a39beda59:/# apt update
apt install -y docker.io
Get:1 http://deb.debian.org/debian trixie InRelease [140 kB]
Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9671 kB]
Get:5 http://deb.debian.org/debian trixie-updates/main amd64 Packages [5412 B]
Get:6 http://deb.debian.org/debian-security trixie-security/main amd64 Packages [127 kB]
Fetched 10.0 MB in 2s (4100 kB/s)
2 packages can be upgraded. Run 'apt list --upgradable' to see them.
Installing:
  docker.io

Installing dependencies:
  apparmor                libapparmor1        libmnl0                 libpython3.13-minimal   python3-minimal
  containerd              libbpf1             libmodule-find-perl     libpython3.13-stdlib    python3-protobuf
  criu                    libcap2-bin         libnet1                 libreadline8t64         python3-pycriu
  dbus                    libcompel1          libnetfilter-conntrack3 libsort-naturally-perl  python3.13
  dbus-bin                libcryptsetup12     libnfnetlink0           libsystemd-shared       python3.13-minimal
  dbus-daemon             libdbus-1-3         libnftables1            libterm-readkey-perl    readline-common
  dbus-session-bus-common libdevmapper1.02.1  libnftnl11              libtirpc-common         runc
  dbus-system-bus-common  libelf1t64          libnl-3-200             libtirpc3t64            systemd
  dbus-user-session       libintl-perl        libnss-systemd          libxtables12            systemd-cryptsetup
  dmsetup                 libintl-xs-perl     libpam-cap              linux-sysctl-defaults   systemd-sysv
  docker-buildx           libip4tc2           libpam-systemd          media-types             systemd-timesyncd
  docker-cli              libip6tc2           libproc-processtable-perl needrestart       xz-utils
  gettext-base            libjansson4         libprotobuf-c1          netbase
  iproute2                libjson-c5          libprotobuf32t64        nftables
```





9. Part E: Execution Flow (Stage-wise Explanation)

- Stage 1: Code Push - Developer updates code in GitHub
- Stage 2: Webhook Trigger - GitHub sends event to Jenkins

- Stage 3: Jenkins Pipeline Execution
- Stage: Clone - Pulls latest code from GitHub
- Stage: Build - Docker builds image using Dockerfile
- Stage: Auth - Jenkins logs into Docker Hub using stored token
- Stage: Push - Image pushed to Docker Hub
- Stage 4: Artifact Ready - Docker image available globally

10. Role of Same Host Agent

- Jenkins runs inside Docker
- Docker socket mounted: /var/run/docker.sock
- Effect: Jenkins directly controls host Docker. Builds and pushes images without separate agent.

11. Observations

- Jenkins GUI simplifies CI/CD management
- GitHub acts as source + pipeline definition
- Docker ensures consistent builds
- Webhook enables automation

12. Result

Successfully implemented a complete CI/CD pipeline where:
* Source code and pipeline are maintained in GitHub
* Jenkins automatically detects changes
* Docker image is built on host agent
* Image is securely pushed to Docker Hub

13. Viva Questions

1. What is the role of Jenkinsfile?
2. How does Jenkins integrate with GitHub?
3. Why is Docker used in CI/CD?
4. What is a webhook?
5. Why store Docker Hub token in Jenkins credentials?
6. What is the benefit of using same host as agent?

14. Key Notes

- Jenkins is GUI-based but pipeline is code-driven

- Always use credentials store (never hardcode secrets)
- Webhook makes CI/CD fully automatic
- This setup is ideal for learning and small deployments

Understanding Jenkins Pipeline Syntax (Simplified Explanation)

Jenkins pipelines (written in a Jenkinsfile) follow a clear structure of blocks and steps.

1. Basic Pipeline Structure

```
pipeline {
agent any
stages {
stage('Build') {
steps {
sh 'echo Hello'
}
}
}
}
```

2. Key Terms Explained
   * pipeline {} - Root block. Everything is written inside this.
   * agent any - Defines where the pipeline runs (any available node).
   * stages {} - Groups all phases of pipeline. Logical separation (Clone, Build, Test, Deploy).
   * stage('Name') - A single step/phase in pipeline. Visible in Jenkins GUI as blocks.
   * steps {} - Contains actual commands to execute.

Common Steps
* git 'https://github.com/user/repo.git' - Clones source code from GitHub
* sh 'echo Hello' - Runs Linux commands inside agent. Equivalent to terminal command.
* echo "Build started" - Prints message in Jenkins console log.

3. The Challenging Part: withCredentials
   You need to login to Docker Hub, but you should NOT write password directly in code. Jenkins stores it securely.

Step-by-Step Meaning:
1. Store Secret in Jenkins with ID: dockerhub-token
2. withCredentials Block temporarily injects secret into environment variable.
string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')
* string = Type of secret (text token)
* credentialsId = Name used in Jenkins
* variable = Temporary variable name
3. What Happens Internally: Jenkins does DOCKER_TOKEN=your_actual_secret_value
4. Using It in sh: sh 'echo $DOCKER_TOKEN | docker login -u username --password-stdin'

Simplified Analogy:
* Locker (Jenkins Credentials) -> You ask by ID (dockerhub-token) -> Jenkins gives temporary key

(DOCKER_TOKEN) -> You use it then it disappears.

4. Key Takeaways
    * pipeline -> stages -> stage -> steps = structure
    * sh = run commands
    * git = fetch code
    * withCredentials = securely use secrets
    * Secrets are temporary and protected